Cours 3

Fichiers, chemins et programme en ligne de commande

1re année géomatique
2027-2028

Objectifs de la séance



Manipuler des chemins et lire des fichiers en Python.
Construire un programme en ligne de commande en Python.

Vu aux cours 1 et 2	Aujourd'hui, en Python
chemin relatif, chemin absolu	`Path`, `/`, `resolve()`, `Path.cwd()`
fichier texte	`open(chemin, encoding="utf-8")`, `f.write(texte)`
disque et mémoire vive	`f.read()`, du disque à la mémoire vive
lancer un programme au terminal	`argparse`, dans un programme qu'on écrit

Portabilité du code

Faciliter l’utilisation de son code par une autre personne :
chemin de fichier, et interface en ligne de commande

Pb 1 : code et données dans le même fichier

Avantage : le script s'exécute sans erreur sur tous les postes.
Inconvénient : il faut modifier le script pour traiter d'autres données.

altitudes.py

```
altitudes = [79.6, 103.2, 98.3, 92.5, 123.7]

print("minimum :", min(altitudes))
print("maximum :", max(altitudes))
```

Poste d'Alice `C:/Users/alice/cours3/altitudes.py`

```
$ python altitudes.py
minimum : 79.6
maximum : 123.7
```

↓ copie du script

Autre poste `C:/Users/eleve/projet/altitudes.py`

```
$ python altitudes.py
minimum : 79.6
maximum : 123.7
```

Pour traiter d'autres données. Ouvrir `altitudes.py`, puis remplacer les valeurs de la liste `altitudes`.

Pb 2 : chemin absolu au milieu du code

Les données sont dans un fichier, séparé du code.

`C:/Users/alice/…` n'existe pas ailleurs, ni après un déplacement du dossier.

altitudes.py

```
altitudes = []
with open("C:/Users/alice/cours3/altitudes.txt",
          encoding="utf-8") as f:
    for ligne in f:
        altitudes.append(float(ligne))

print("minimum :", min(altitudes))
print("maximum :", max(altitudes))
```

altitudes.txt

```
79.6
…
123.7
```

Poste d'Alice `C:/Users/alice/cours3/`

`altitudes.py, altitudes.txt`

```
$ python altitudes.py
minimum : 79.6
maximum : 123.7
```

↓ copie du dossier

Autre poste `C:/Users/eleve/projet/`

`altitudes.py, altitudes.txt`

```
$ python altitudes.py
FileNotFoundError: [Errno 2] No such file or
directory: 'C:/Users/alice/cours3/altitudes.txt'
```

Pour traiter d'autres données. Ouvrir `altitudes.py`, chercher le chemin dans le code, puis le remplacer.

Pb 3 : chemin relatif écrit dans le script

Le chemin est relatif au dossier du script et ne dépend plus du poste.
Une autre personne trouve le nom du fichier au début du script, sans lire le reste du code.

altitudes.py

```python
from pathlib import Path

DOSSIER = Path(__file__).parent
FICHIER = DOSSIER / "altitudes.txt"

altitudes = []
with open(FICHIER, encoding="utf-8") as f:
    for ligne in f:
        altitudes.append(float(ligne))

print("minimum :", min(altitudes))
print("maximum :", max(altitudes))
```

Poste d'Alice `C:/Users/alice/cours3/`

`altitudes.py, altitudes.txt`

```
$ python altitudes.py
minimum : 79.6
maximum : 123.7
```

↓ copie du dossier

Autre poste `C:/Users/eleve/projet/`

`altitudes.py, altitudes.txt`

```
$ python altitudes.py
minimum : 79.6
maximum : 123.7
```

Pour traiter d'autres données. Copier son fichier dans le dossier du script. Ouvrir `altitudes.py`, puis remplacer `altitudes.txt` par le nom de son fichier.

Solution : chemin donné en ligne de commande

Le code ne contient plus de nom de fichier.

altitudes.py

```
import argparse

analyseur = argparse.ArgumentParser()
analyseur.add_argument("fichier")
options = analyseur.parse_args()

altitudes = []
with open(options.fichier, encoding="utf-8") as f:
    for ligne in f:
        altitudes.append(float(ligne))

print("minimum :", min(altitudes))
print("maximum :", max(altitudes))
```

Poste d'Alice `C:/Users/alice/cours3/`

`altitudes.py, altitudes.txt`

```
$ python altitudes.py altitudes.txt
minimum : 79.6
maximum : 123.7
```

↓ copie du script

Autre poste `C:/Users/eleve/projet/`

`altitudes.py, mes_altitudes.txt`

```
$ python altitudes.py mes_altitudes.txt
minimum : 111.8
maximum : 118.9
```

Pour traiter d'autres données. Écrire le chemin de son fichier après le nom du script. Le code ne change pas.

Récapitulatif des quatre versions

Version	Sur un autre poste	Pour traiter d'autres données
code et données dans le même fichier	le script s'exécute sans erreur	remplacer les valeurs de la liste dans le script
chemin absolu au milieu du code	le script s'arrête sur `FileNotFoundError`	chercher le chemin dans le code, puis le remplacer
chemin relatif écrit dans le script	le script s'exécute sans erreur	copier le fichier dans le dossier du script, puis remplacer `altitudes.txt` au début du script
chemin donné en ligne de commande	le script s'exécute sans erreur	écrire le chemin après le nom du script, sans modifier le code

Autres bonnes pratiques de développement

D'autres problèmes de portabilité et d'autres bonnes pratiques de développement ne sont pas traités dans cet enseignement. Python est dit « batteries included » (piles fournies), car sa bibliothèque standard, installée avec lui, fournit des modules pour beaucoup d'entre eux.

Sujet	Intérêt	Module
fichier de configuration	regrouper les réglages dans un fichier, séparé du code	`configparser`
dossier personnel	trouver le dossier de l'utilisateur sous Windows, Linux ou macOS	`pathlib`
journal (log)	enregistrer ce que le programme a fait, par exemple le fichier lu	`logging`
tests	vérifier qu'une fonction renvoie le résultat attendu	`unittest`

Bibliothèque standard de Python

Les quatre versions du script emploient `open`, `pathlib` et `argparse`, détaillés dans la partie suivante. Comme `pathlib` et `argparse`, d'autres modules de la bibliothèque standard s'importent sans installation.

Partie suivante

`open`	ouvrir un fichier pour le lire ou l'écrire
`pathlib`	construire un chemin à partir du dossier du script
`argparse`	lire les arguments donnés en ligne de commande

Autres modules utiles

`csv`	lire et écrire un fichier CSV
`statistics`	calculer une moyenne, une médiane
`math`	calculer une racine carrée, un logarithme
`datetime`	lire une date, calculer une durée

Avant d'écrire une fonction, chercher si un module la fournit déjà. Liste complète : docs.python.org/fr/3/library/.

Fichiers et encodage

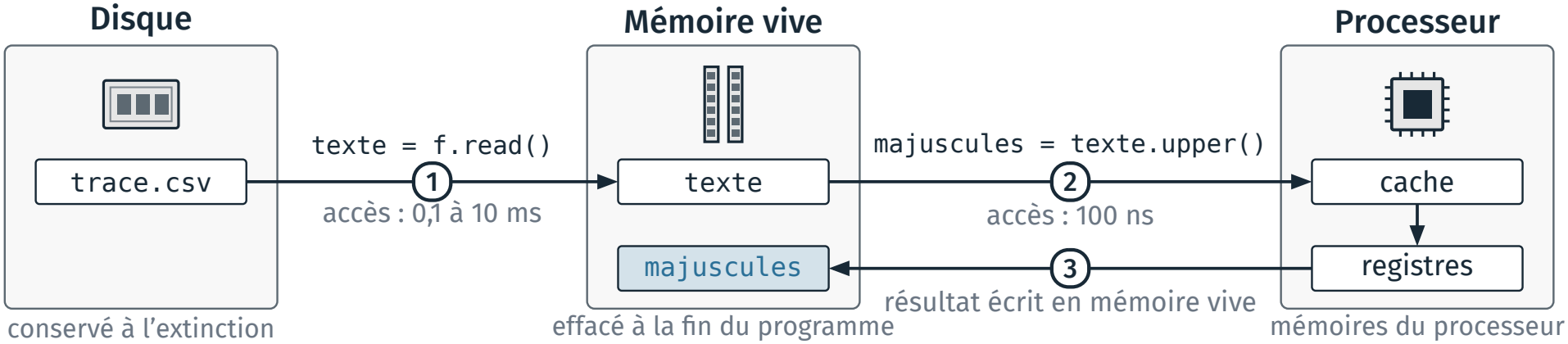
Lire un fichier ; stockage des fichiers et mémoire ; fichier texte ou binaire et encodage ; écrire un fichier ; pathlib et argparse

Lecture d’un fichier : du disque au processeur

Lire un fichier demande trois étapes : l’ouvrir avec la fonction `open`, le lire avec une méthode de l’objet fichier, puis le fermer avec la méthode `close`. Le programme traite ensuite le texte lu.

```
f = open(chemin, encoding="utf-8")
texte = f.read()
f.close()
majuscules = texte.upper()
```

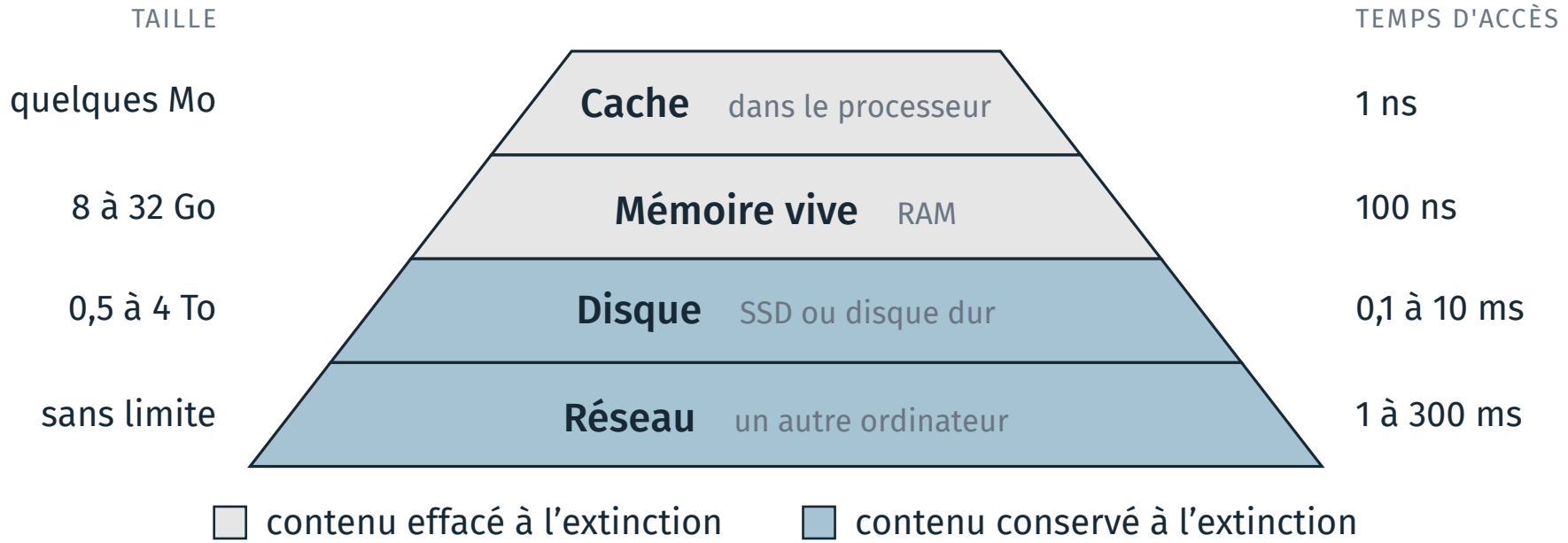
① ouvrir : `f` est l’objet fichier
① lire : copie du fichier du disque vers la mémoire vive
② ③ fermer
② ③ traiter : le processeur copie `texte` dans son cache



Une modification de `texte` reste en mémoire vive. Le fichier sur le disque ne change qu’avec `f.write`.

Hiérarchie des mémoires

Les mémoires sont rangées selon leur proximité du processeur, la plus proche en haut. La mémoire vive est rapide et s'efface à l'extinction ; le disque est lent et conserve son contenu.



La mémoire vive contient les variables d'un programme en cours. Le disque contient les fichiers et les programmes installés.

Ordres de grandeur : tailles et temps d’accès

1 octet
8 bits
une lettre : a

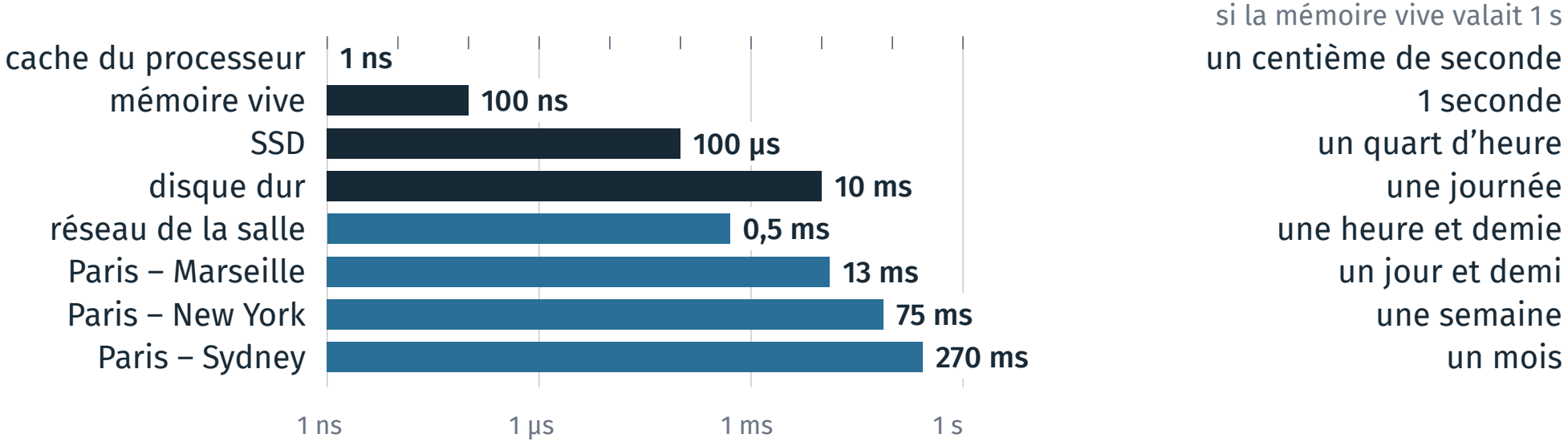
1 Ko
le poème du cours 1 : 1,3 Ko

1 Mo
une photo de téléphone : 3 Mo

100 Mo
une dalle d’orthophoto : 75 Mo

1 Go
une heure de vidéo HD : 1 à 3 Go

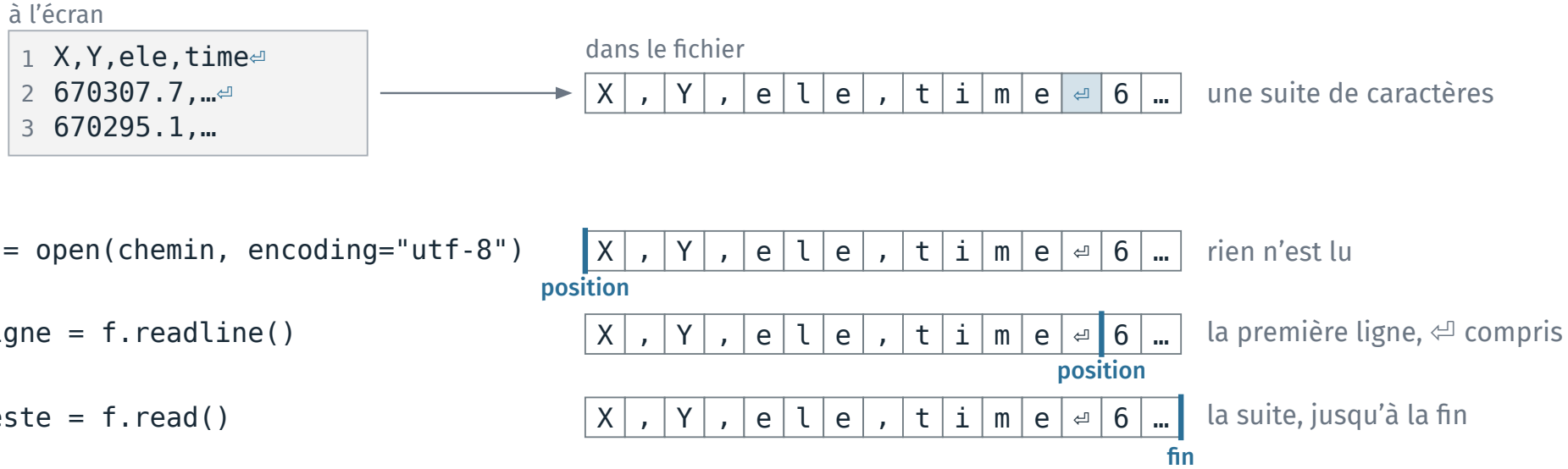
1 To
le disque d’un portable



Chaque unité de taille vaut mille fois la précédente. Temps d’un accès ou d’un aller-retour sur une échelle logarithmique : chaque graduation vaut dix fois la précédente. Réseau mesuré depuis Paris (wondernetwork.com, septembre 2026).

En mémoire : fichier texte == suite de caractères

Un éditeur affiche le texte sur plusieurs lignes. Dans le fichier, le texte est une seule suite de caractères, et chaque fin de ligne est un caractère, noté ⏎ ci-dessous. L'objet fichier contient une position de lecture. Chaque lecture part de cette position et la fait avancer.



La méthode `readline` lit jusqu'au prochain ⏎, inclus.

Boucle sur un fichier : une ligne par itération

Même syntaxe que la boucle sur une liste, vue au cours 1.

Liste `altitudes`

```
altitudes = [79.6, 103.2, 98.3, 92.5, 123.7]
for altitude in altitudes:
    print(altitude)
```

Affichage

```
79.6
103.2
98.3
92.5
123.7
```

Fichier `altitudes.txt`

```
f = open("altitudes.txt", encoding="utf-8")
for ligne in f:
    print(ligne)
f.close()
```

Affichage

```
79.6

103.2

98.3

92.5

123.7

```

Une ligne vide suit chaque valeur, car `ligne` garde son ⏎ et `print` écrit une fin de ligne de plus. `ligne.strip()` retire le ⏎.

Fin de ligne

La fin de ligne est un caractère de contrôle, écrit `\n` en Python. Sous Linux et macOS, elle est codée par ce seul caractère ; sous Windows, par les deux caractères `\r\n`.

Linux, macOS	F	o	r	ê	t	,	2		k	m	\n	É	12 caractères
texte lu	F	o	r	ê	t	,	2		k	m	\n	É	

Windows	F	o	r	ê	t	,	2		k	m	\r	\n	É	13 caractères
texte lu	F	o	r	ê	t	,	2		k	m	\n		É	

dans les deux cas, la chaîne `'Forêt,2 km\nÉ'`

En mode texte, Python remplace `\r\n` par `\n` à la lecture. À l'écriture sous Windows, il remplace chaque `\n` par `\r\n`. Dans le programme, les lignes sont toujours séparées par `\n`.

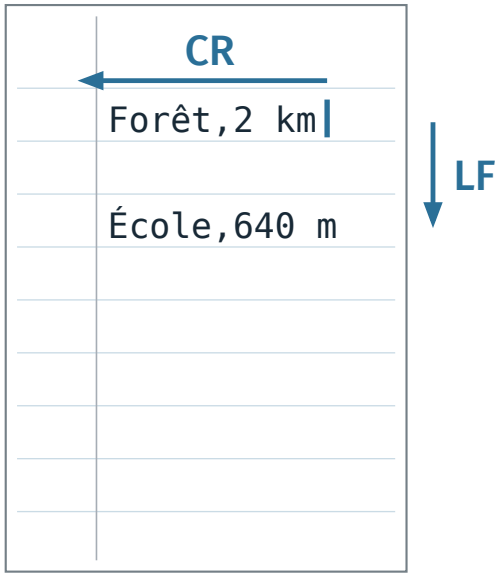


VS Code affiche la fin de ligne du fichier dans la barre d'état : `LF` pour `\n`, `CRLF` pour `\r\n`. Pour en changer, cliquer sur cette mention.

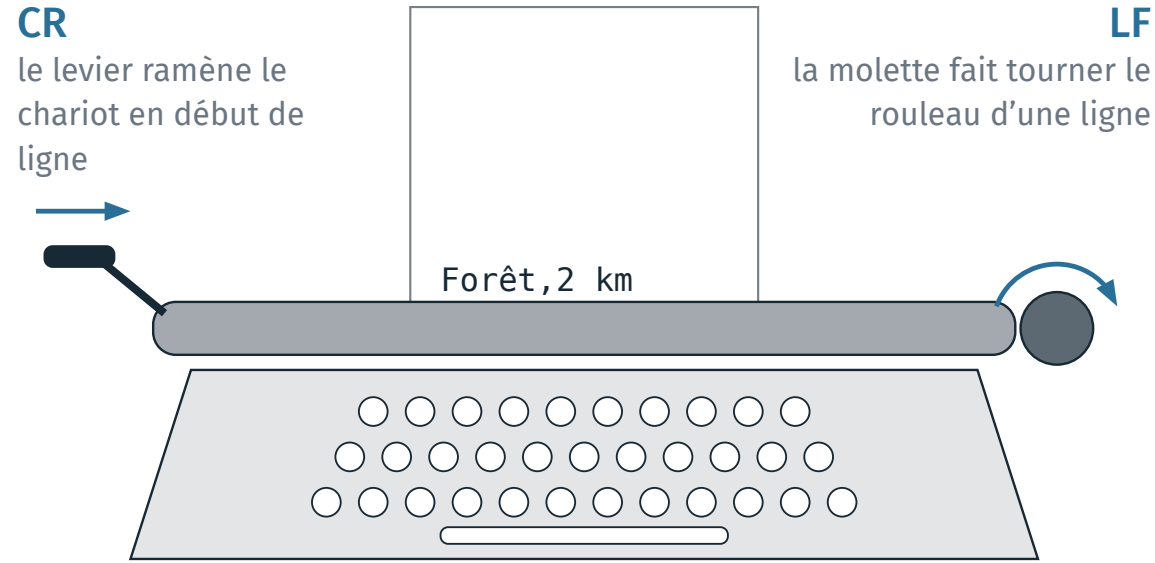
Origine de CR et LF : la machine à écrire

Les téléscripteurs, puis le code ASCII, ont codé les deux mouvements du passage à la ligne par deux caractères de contrôle : CR, *carriage return* (retour chariot), et LF, *line feed* (saut de ligne).

Sur la feuille



Sur la machine à écrire



with : fermeture automatique

Un fichier ouvert doit être fermé. Le bloc `with` le ferme à la sortie du bloc indenté, y compris en cas d'erreur.

open, puis close

```
f = open(chemin, encoding="utf-8")
texte = f.read()
f.close()
```

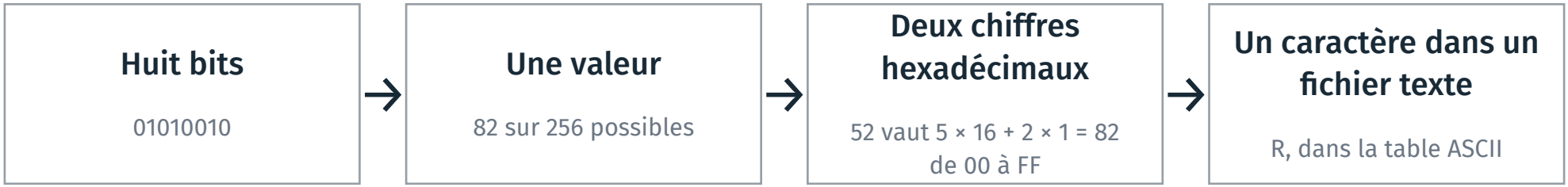
with

```
with open(chemin, encoding="utf-8") as f:
    texte = f.read()
# ici, f est fermé
```

Situation	`open`, puis `close()`	`with`
une erreur survient pendant la lecture	le fichier reste ouvert, car `close()` n'est pas exécuté	fermé
`close()` oublié	reste ouvert jusqu'à la fin du programme	aucun `close()` à écrire

Fichiers binaires et fichiers texte

Un fichier est une suite de **bits**, 0 ou 1, groupés par huit en **octets**. Un octet a 256 valeurs possibles, notées par deux chiffres hexadécimaux. Dans un fichier **texte**, les octets codent des caractères. Dans un fichier **binaire**, ils codent des données selon un format.

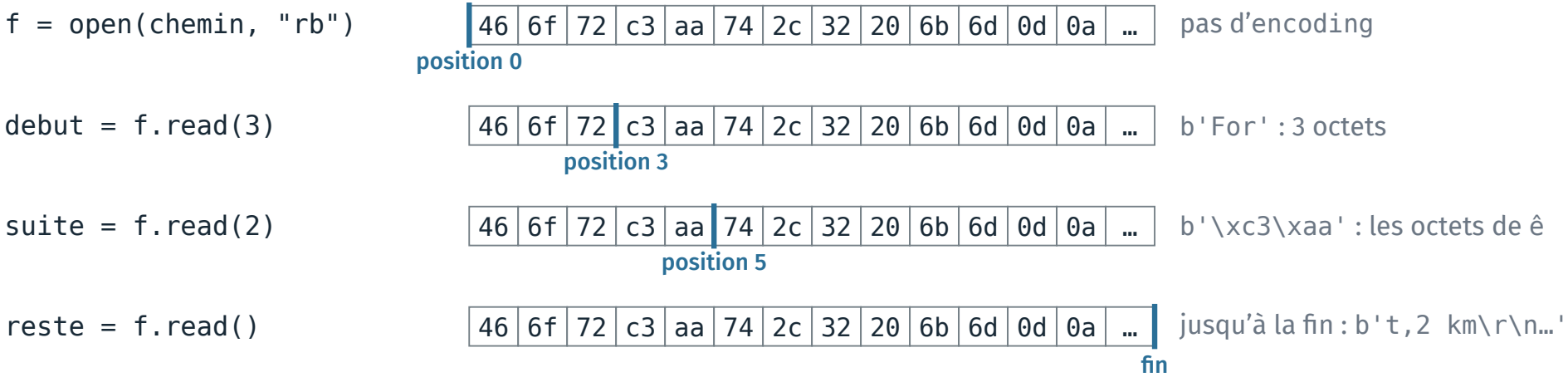


	Fichier texte	Fichier binaire
Exemples	`.txt`, `.csv`, `.py`	`.png`, `.odt`, `.exe`
Lecture	dans un éditeur de texte	dans un logiciel adapté au format
`git diff`	les lignes modifiées	`Binary files … differ`

$2^8 = 256$. L'hexadécimal emploie seize chiffres, de `0` à `F`.

Lecture en mode binaire

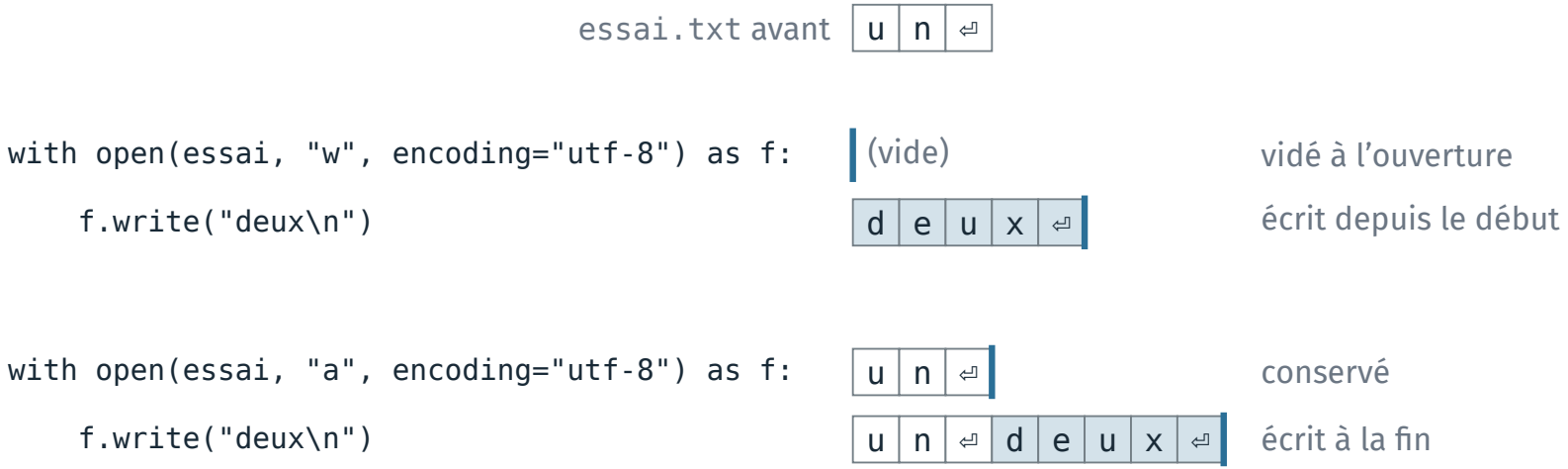
Le deuxième argument de `open` est le mode d'ouverture. Avec le mode `"rb"`, Python lit les octets du fichier sans les convertir en caractères : `read` renvoie un objet `bytes`, et les fins de ligne restent `\r\n`. La position est comptée en octets.



Le mode `"wb"` écrit des octets : `f.write(b"...")`. Un programme lit et écrit une image en mode binaire.

Modes d’écriture : "w" et "a"

En mode `"w"`, Python vide le fichier à l’ouverture, puis écrit depuis le début. En mode `"a"`, Python conserve le contenu et écrit à la fin. Dans les deux modes, un fichier absent est créé.



Le trait bleu marque la position d’écriture ; les cases en couleur sont les caractères écrits par `write`.

Encodage d’un fichier texte

L’encodage, donné par l’argument `encoding` de `open`, définit le caractère qui correspond à chaque octet ou groupe d’octets. Décodés avec un autre encodage, les mêmes octets donnent d’autres caractères.

octets du fichier	46	6f	72	c3	aa	74	2c	32	20	6b	6d	0a
												12 octets
`encoding="utf-8"`	F	o	r	ê		t	,	2		k	m	⏎
												11 caractères : c3 aa donne ê
`encoding="cp1252"`	F	o	r	Ã	ª	t	,	2		k	m	⏎
												12 caractères : c3 donne Ã, aa donne ª

⏎ représente le retour à la ligne. Sans l’argument `encoding`, Python sous Windows décode en cp1252, et `Forêt` s’affiche `ForÃªt`.

Encodage texte : ASCII et UTF-8

Un encodage associe à chaque caractère un ou plusieurs octets. ASCII code 128 caractères sur un octet : lettres sans accent, chiffres, ponctuation. UTF-8 code ces 128 caractères de la même façon, et les autres sur deux à quatre octets.

Quatre caractères

Caractère	ASCII	UTF-8
a	61	61
é	—	c3 a9
œ	—	c5 93
😀	—	f0 9f 98 80

Un mot, avec et sans ligature

Mot	len	UTF-8
œuf	3	c5 93 75 66, 4 octets
oeuf	4	6f 65 75 66, 4 octets

```
"œuf".encode("utf-8").hex(" ")
"œ".encode("ascii")
```

`encode` renvoie les octets, `hex` les affiche en hexadécimal
lève `UnicodeEncodeError` : œ n'a pas de code ASCII

Fichiers en Python : récapitulatif

Ouvrir

```
with open(chemin, mode, encoding="utf-8") as f:
```

`"r"`	lire du texte (mode par défaut)
`"w"`	vider le fichier, puis écrire
`"a"`	écrire à la fin du fichier
`"rb"`, `"wb"`	lire, écrire des octets, sans `encoding`

Lire

`f.read()`	une chaîne : tout le fichier
`f.readline()`	une chaîne : la ligne suivante
`f.readlines()`	une liste de chaînes, une par ligne
`for ligne in f:`	une chaîne à chaque itération

Écrire

`f.write(texte)`	écrit la chaîne, sans fin de ligne

Traiter une ligne

`ligne.strip()`	enlève la fin de ligne
`ligne.split(",")`	découpe la ligne aux virgules

Exemple, sur `trace.csv`

```python
# Lire une ligne à la fois
altitudes = []
with open("trace.csv", encoding="utf-8") as f:
    f.readline()
    for ligne in f:
        x, y, ele, time = ligne.strip().split(",")
        altitudes.append(float(ele))

# Vider le fichier, puis écrire
with open("resume.txt", "w", encoding="utf-8") as sortie:
    sortie.write(f"minimum : {min(altitudes)}\n")

# Écrire à la fin du fichier
with open("resume.txt", "a", encoding="utf-8") as sortie:
    sortie.write(f"maximum : {max(altitudes)}\n")
```

`resume.txt`, après l'exécution

```
minimum : 78.8
maximum : 129.5
```

Chemin relatif : le dossier du terminal

Un chemin relatif écrit dans un script part du dossier courant du terminal, comme les chemins donnés à `ls` et à `cp` au cours 1.

altitudes.py

```
altitudes = []
with open("trace.csv", encoding="utf-8") as fichier:
    fichier.readline()
    for ligne in fichier:
        …
```

Dossier du terminal …/3b_portabilite/travail/

altitudes.py, trace.csv

```
$ python altitudes.py
altitude minimale : 78.8 m
altitude maximale : 129.5 m
```

↓ `cd ..`

Dossier du terminal …/3b_portabilite/

depart/, travail/

```
$ python travail/altitudes.py
FileNotFoundError: [Errno 2] No such file or
directory: 'trace.csv'
```

Objet Path : parties d'un chemin

La classe `Path`, importée de `pathlib`, transforme une chaîne en objet chemin. L'objet donne chaque partie du chemin, et l'opérateur `/` lui ajoute un dossier ou un fichier.




```
Path("C:/Users/alice/cours3") / "trace.csv"
chemin.parent / "resume.txt"
chemin.exists()
```

construit le même chemin avec l'opérateur `/`
construit `C:/Users/alice/cours3/resume.txt`
vérifie que le fichier existe : `True` ou `False`

Dossier du script : `__file__`

Au lancement d'un script, Python enregistre le chemin absolu du script dans la variable `__file__`. `Path(__file__).parent` donne le dossier du script, quel que soit le dossier du terminal.

```
from pathlib import Path
print("dossier du terminal :", Path.cwd())
print("dossier du script :", Path(__file__).parent)
```

Commande	Dossier du terminal	Dossier du script
`python altitudes.py`	`…\3b_portabilite\travail`	`…\3b_portabilite\travail`
`python travail/altitudes.py`	`…\3b_portabilite`	`…\3b_portabilite\travail`

… : le dossier du cours sur le poste, par exemple `C:\Users\eleve\Desktop\info01\cours3`.

pathlib : un chemin à partir du script

Un seul chemin est déclaré : le dossier du script, déduit de `__file__`. Les autres chemins sont construits à partir de lui avec l'opérateur `/`.

L'arborescence du TD 3b

```
3b_portabilite/
└── travail/            ← DOSSIER
    ├── altitudes.py    ← Path(__file__)
    └── trace.csv       ← FICHIER
```

Le code

```
from pathlib import Path

DOSSIER = Path(__file__).parent
FICHIER = DOSSIER / "trace.csv"
```

pathlib : récapitulatif

La bibliothèque `pathlib` manipule les chemins en Python. Les fonctions ci-dessous sont parmi les plus courantes ; la documentation décrit les autres, comme `glob`, `with_suffix` ou `read_text` : docs.python.org/fr/3/library/pathlib.html.

```
from pathlib import Path                          la classe des chemins
Path("C:/Users/alice/cours3")                     déclare un chemin ; ne crée et ne vérifie rien
Path.cwd()                                        le dossier courant
Path(__file__)                                    le chemin du script
dossier / "trace.csv"                             ajoute un dossier ou un fichier au chemin

chemin.name, chemin.stem, chemin.suffix           trace.csv, trace, .csv
chemin.parent                                     le dossier qui contient le chemin

chemin.exists(), chemin.is_dir(), chemin.is_file()  existe ; est un dossier ; est un fichier
dossier.iterdir()                                 les fichiers et sous-dossiers, sans ordre
chemin.resolve()                                  le chemin absolu, sans ..
chemin.relative_to(racine)                        le chemin à partir de racine
```

Arguments positionnels et options

Les arguments sont les mots écrits après le nom du script.

```
python altitudes.py trace.csv -s 2
```

le programme — le script — argument positionnel — option et sa valeur

	Argument positionnel	**Option**
Écriture	sans tiret : `trace.csv`	avec un ou deux tirets : `-s 2`, `--seuil 2`
Reconnu par	sa place sur la ligne	son nom
Si absent	une erreur, car il est obligatoire	une valeur par défaut
Au cours 1	`cp source destination`	`ls -a`, `cp -r`

Arguments d'un script : la liste `sys.argv`

Python enregistre les mots écrits après le nom du script dans la liste `sys.argv`, sous forme de chaînes, dans l'ordre de la frappe. Le premier élément est le nom du script.

altitudes.py

```
import sys

print(sys.argv)
```

```
$ python altitudes.py
['altitudes.py']
$ python altitudes.py trace.csv
['altitudes.py', 'trace.csv']
$ python altitudes.py trace.csv -s 2
['altitudes.py', 'trace.csv', '-s', '2']
$ python altitudes.py -s 2 trace.csv
['altitudes.py', '-s', '2', 'trace.csv']
```

Commande	sys.argv[0]	sys.argv[1]	sys.argv[2]	sys.argv[3]
python altitudes.py trace.csv -s 2	'altitudes.py'	'trace.csv'	'-s'	'2'

Lecture de `sys.argv` par position : limites

Lire les arguments à des positions fixes impose leur ordre et leur nombre. Les messages de Python décrivent l'erreur dans le code, sans indiquer à l'utilisateur ce qu'il doit taper.

```
import sys
fichier = sys.argv[1]
seuil = float(sys.argv[3])
```

Après `python altitudes.py`	**Résultat**	**Cause**
`trace.csv -s 2`	`fichier` vaut `'trace.csv'`, `seuil` vaut `2.0`	chaque argument est à sa place
`trace.csv`	`IndexError: list index out of range`	`sys.argv[3]` n'existe pas
`-s 2 trace.csv`	`ValueError: could not convert string to float: 'trace.csv'`	`sys.argv[3]` vaut `'trace.csv'`
`--help`	`IndexError: list index out of range`	`sys.argv[1]` vaut `'--help'`, et `sys.argv[3]` n'existe pas

argparse : déclaration et lecture des arguments

`argparse`, bibliothèque standard, lit les arguments écrits après le nom du script : il les vérifie, les convertit, et construit l'aide de `--help`.

```
import argparse                                                          la bibliothèque, à importer
analyseur = argparse.ArgumentParser()                                    l'analyseur des arguments
analyseur.add_argument("fichier")                                        obligatoire
analyseur.add_argument("-s", "--seuil", type=float, default=0)           option ; convertie en nombre ; 0 si absente
options = analyseur.parse_args()                                         lit la ligne de commande
```

Commande	`options.fichier`	`options.seuil`
`python altitudes.py trace.csv`	`'trace.csv'`	`0`
`python altitudes.py trace.csv -s 2`	`'trace.csv'`	`2.0`
`python altitudes.py -s 2 trace.csv`	`'trace.csv'`	`2.0`

`add_argument("fichier")` crée `options.fichier`, et `"--seuil"` crée `options.seuil`. `parse_args` renvoie un objet avec un attribut par argument ; `vars(options)` le transforme en dictionnaire.

argparse : aide et messages d'erreur

```
analyseur = argparse.ArgumentParser(description="Altitudes et cumul d'une trace GPS.")
analyseur.add_argument("fichier", help="le fichier CSV de la trace")
analyseur.add_argument("-s", "--seuil", type=float, default=0,
                       help="variation ignorée sous ce seuil, en mètres")
```

Aide

```
$ python altitudes.py --help
usage: altitudes.py [-h] [-s SEUIL] fichier

Altitudes et cumul d'une trace GPS.

positional arguments:
  fichier            le fichier CSV de la trace

options:
  -h, --help         show this help message and exit
  -s, --seuil SEUIL  variation ignorée sous ce seuil, en mètres
```

Messages d'erreur

```
$ python altitudes.py
altitudes.py: error: the following
arguments are required: fichier

$ python altitudes.py trace.csv -s deux
altitudes.py: error: argument -s/--seuil:
invalid float value: 'deux'
```

Les textes de `description` et de `help` apparaissent dans l'aide. Documentation : docs.python.org/fr/3/library/argparse.html.

Intérêt d'argparse : lecture à la main

Sans argparse, le script parcourt lui-même `sys.argv` pour reconnaître l'option, convertir le seuil et signaler chaque erreur.

Sans argparse

```
import sys
fichier, seuil = None, 0.0
mots = sys.argv[1:]
i = 0
while i < len(mots):
    if mots[i] in ("-h", "--help"):
        print("usage: altitudes.py [-h] [-s SEUIL] fichier")
        sys.exit()
    elif mots[i] in ("-s", "--seuil"):
        if i + 1 == len(mots):
            sys.exit("erreur : -s demande une valeur")
        try:
            seuil = float(mots[i + 1])
        except ValueError:
            sys.exit(f"erreur : seuil invalide : {mots[i + 1]}")
        i += 2
    elif fichier is None:
        fichier = mots[i]
        i += 1
    else:
        sys.exit(f"erreur : argument en trop : {mots[i]}")
if fichier is None:
    sys.exit("erreur : le fichier est obligatoire")
```

Avec argparse

```
import argparse

analyseur = argparse.ArgumentParser()
analyseur.add_argument("fichier")
analyseur.add_argument("-s", "--seuil",
                       type=float, default=0)
options = analyseur.parse_args()
```

argparse accepte en plus `--seuil=2` et `-s2`, que la version de gauche refuse, et produit l'aide complète de `--help`.

TRAVAUX DIRIGÉS

TD 3a	Dénivelé d'une trace GPS `cours3/3a_denivele/`	≈ 20′
TD 3b	Portabilité d'altitudes.py et fusion d'une branche `cours3/3b_portabilite/`	≈ 55′
TD 3c	Les ordres de grandeur de votre poste — facultatif `cours3/3c_mesures/`	≈ 15′

À retenir

Récapituler les chemins, les arguments et les fichiers en Python.

Aide-mémoire : chemins et arguments

Chemins : `pathlib`		**Ligne de commande :** `sys`, `argparse`	
`from pathlib import Path`	importe la classe des chemins	`sys.argv`	liste les arguments, en chaînes
`Path("trace.csv")`	crée un objet chemin	`argparse.ArgumentParser()`	crée l'analyseur
`chemin / "nom"`	ajoute un dossier ou un fichier	`add_argument("fichier")`	déclare un argument obligatoire
`Path(__file__).parent`	donne le dossier du script	`add_argument("-s", default=0)`	déclare une option et sa valeur par défaut
`Path.cwd()`	donne le dossier du terminal	`analyseur.parse_args()`	lit et vérifie les arguments
`.name`, `.suffix`	donnent le nom, l'extension	`options.fichier`	donne la valeur de `fichier`
`chemin.exists()`	vérifie que le chemin existe	`--help`	affiche l'aide construite par `argparse`

Un chemin relatif part du dossier du terminal. Pour un fichier livré avec le script, construire le chemin à partir de `Path(__file__).parent`.

Aide-mémoire : lire et écrire un fichier

Ouvrir		Lire et écrire	
`with open(chemin, mode) as f:`	ouvre le fichier, et le ferme à la fin du bloc	`f.read()`	lit tout le fichier, en une chaîne
`"r"`	lit du texte (mode par défaut)	`f.readline()`	lit la ligne suivante, fin de ligne comprise
`"w"`	vide le fichier, puis écrit	`for ligne in f:`	lit une ligne à chaque tour
`"a"`	écrit à la fin du fichier	`ligne.strip()`	retire la fin de ligne
`"rb"`	lit des octets, sans `encoding`	`ligne.split(",")`	découpe la ligne aux virgules
`f.close()`	ferme le fichier, sans `with`	`f.write(texte)`	écrit la chaîne, sans ajouter `\n`

Chaque lecture part de la position de lecture de l'objet fichier, et la fait avancer.

Aide-mémoire : contenu d’un fichier

Octets et encodage		Fins de ligne et mémoire	
octet	huit bits, 256 valeurs, noté par deux chiffres hexadécimaux : `52`	`\n`, LF	fin de ligne sous Linux et macOS
ASCII	128 caractères, un octet chacun	`\r\n`, CRLF	fin de ligne sous Windows, lue `\n` par Python
UTF-8	un à quatre octets par caractère : ê donne `c3 aa`	barre d’état de VS Code	affiche l’encodage et la fin de ligne du fichier
`encoding="utf-8"`	se donne à chaque `open` en mode texte	disque	conserve les fichiers ; accès en 0,1 à 10 ms
fichier binaire	contient des octets selon un format : `.png`, `.odt`	mémoire vive	garde les variables du programme ; accès en 100 ns